AWS CI/CD Pipeline Configuration

Group: Group 3

Group Members

1. Manu Adam Onyina
2. Othniel Takyi
3. Rahmah Larry
4. Dennis Peprah
5. Joshua Agbozo

Introduction

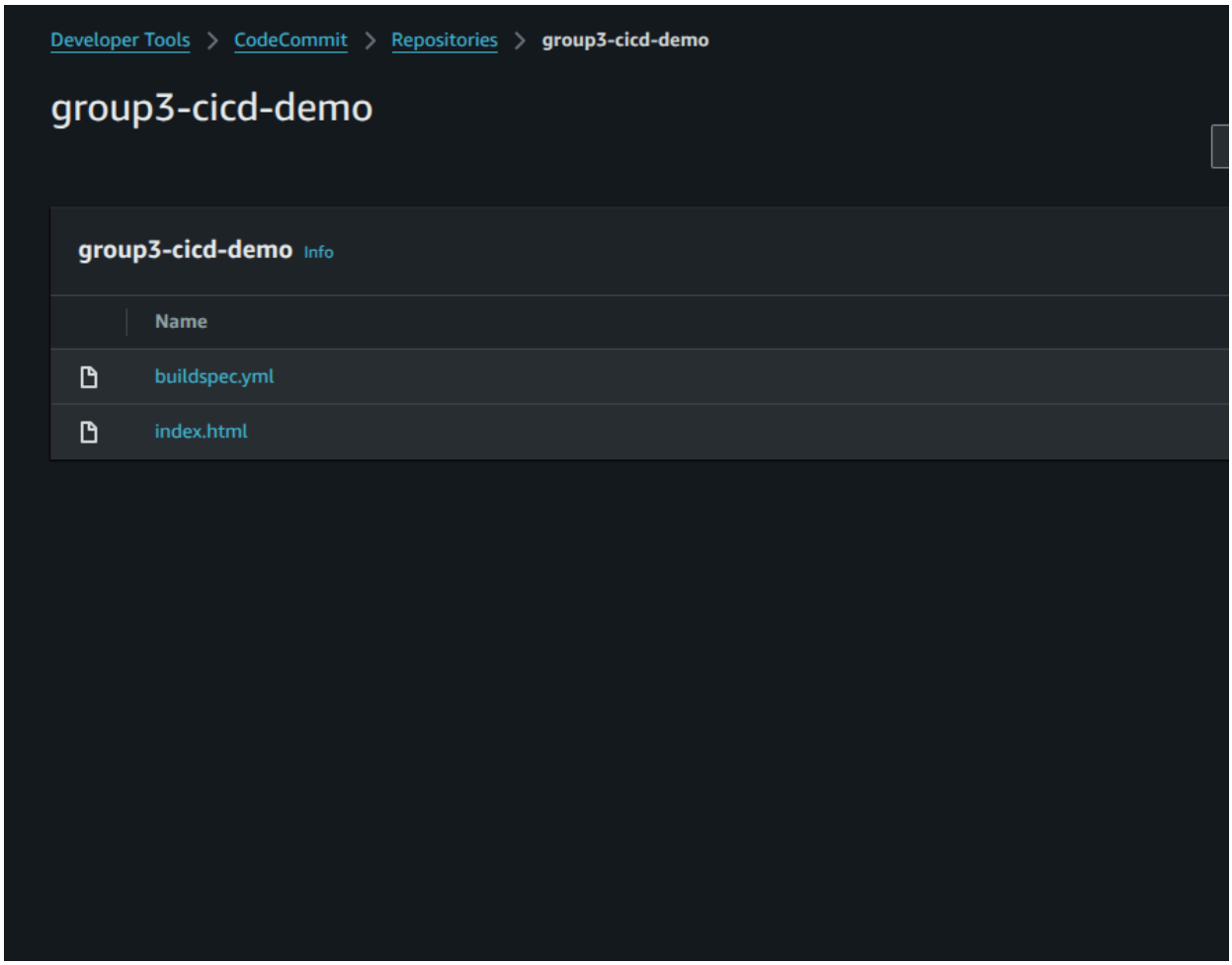
For this group activity, Team 3 successfully designed and built a fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline on AWS. Our goal was to automate the deployment of a static website so that any changes pushed to our code repository would automatically build and deploy to a live web server without any manual intervention.

Here is the step-by-step documentation of our process.

Task 1: Create a CodeCommit Repository (Source Stage)

To start, we needed a secure place to store our application code. We navigated to AWS CodeCommit and created a new repository named group3-cicd-demo.

Once the repository was initialized, we created and committed two essential files: our website's index.html file, and a buildspec.yml file, which tells AWS CodeBuild exactly how to package our application in the next stage.

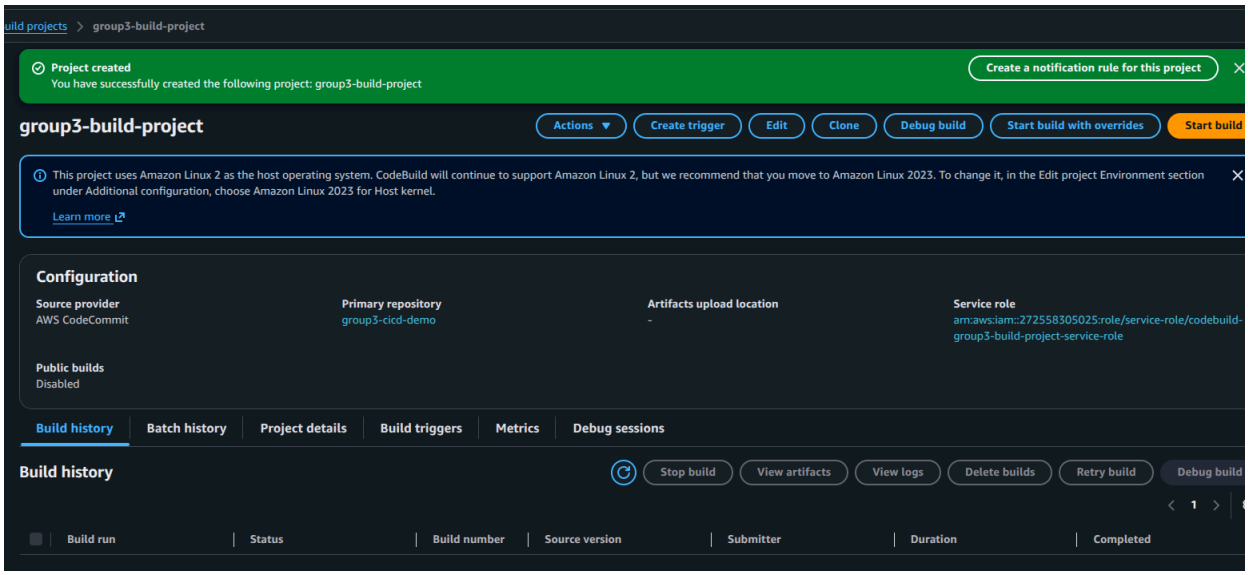


Screenshot showing our CodeCommit repository successfully hosting our initial files

Task 2: Create a Build Project in CodeBuild (Build Stage)

With our source code safely stored, we moved on to the build phase. We opened AWS CodeBuild and created a new project named group3-build-project.

We linked this project directly to our CodeCommit repository and configured the environment to use a standard Ubuntu managed image. This provided a clean, temporary server that uses our buildspec.yml instructions to process the code whenever a change is detected.

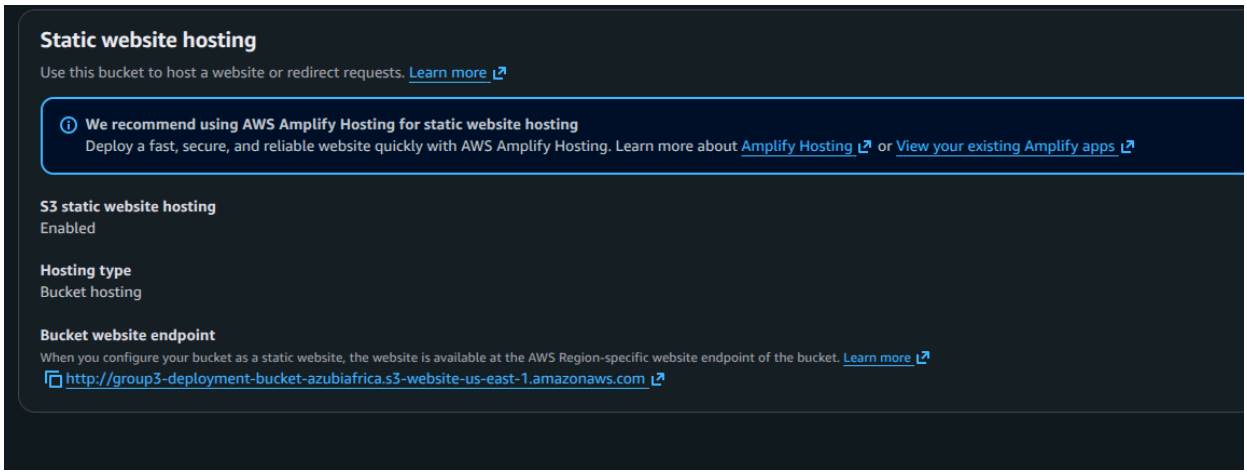


Screenshot showing the successful creation and configuration of our CodeBuild project

Task 3: Create an S3 Bucket for Deployment (Deployment Stage)

Next, we needed a destination to host our live website. We went to Amazon S3 and created a unique bucket named group3-deployment-bucket-azubiafrica.

To make it act like a web server, we disabled the "Block all public access" setting, applied a public-read bucket policy, and enabled **Static website hosting** under the bucket properties. AWS provided us with a dedicated endpoint URL where our site would eventually live.



Screenshot showing Static Website Hosting enabled with our bucket's live endpoint URL

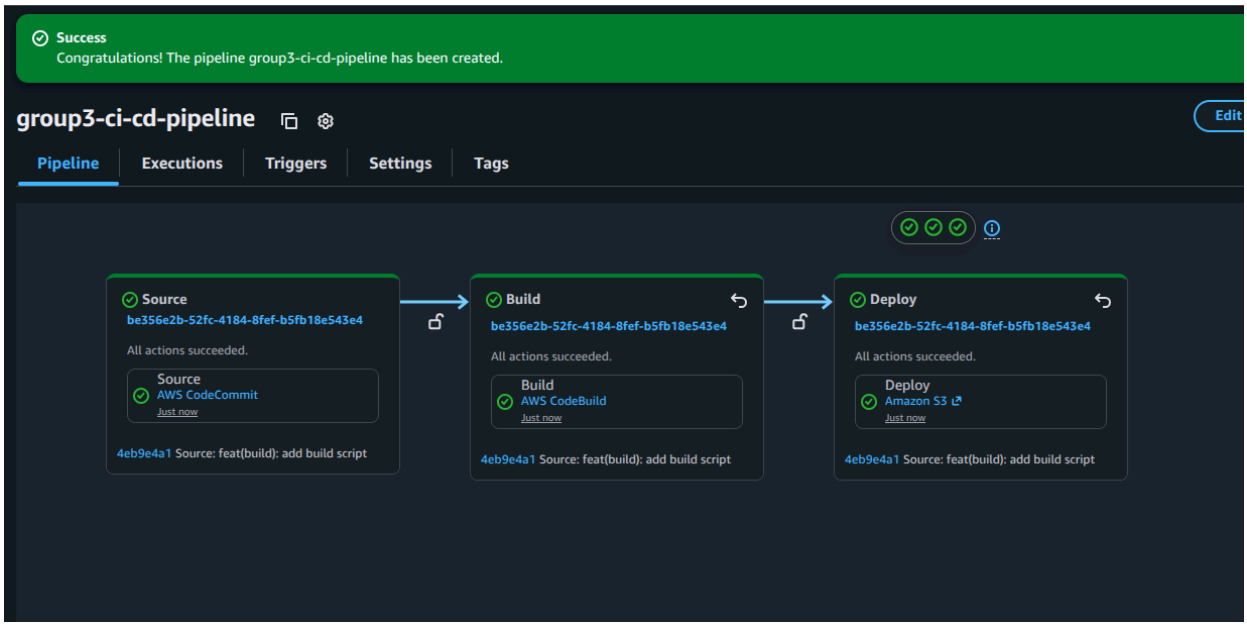
Task 4: Create a Pipeline in AWS CodePipeline

This is where we glued everything together. We navigated to AWS CodePipeline and created group3-ci-cd-pipeline.

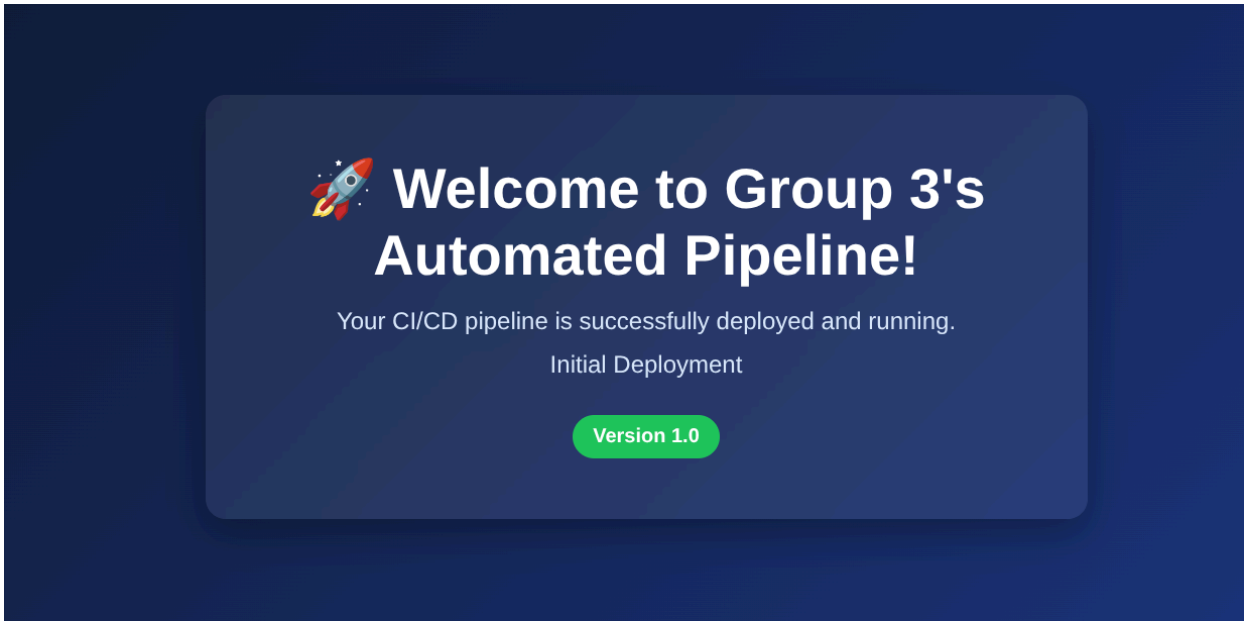
We configured the pipeline with three distinct stages:

1. **Source:** Pulls the latest code from our CodeCommit repo.
2. **Build:** Passes the code to our CodeBuild project.
3. **Deploy:** Extracts the final files and pushes them into our S3 bucket.

As soon as we created the pipeline, it automatically triggered its very first run, successfully deploying our "Version 1.0" website to the internet!



Screenshot showing our CodePipeline running successfully with green checkmarks across Source, Build, and Deploy stages

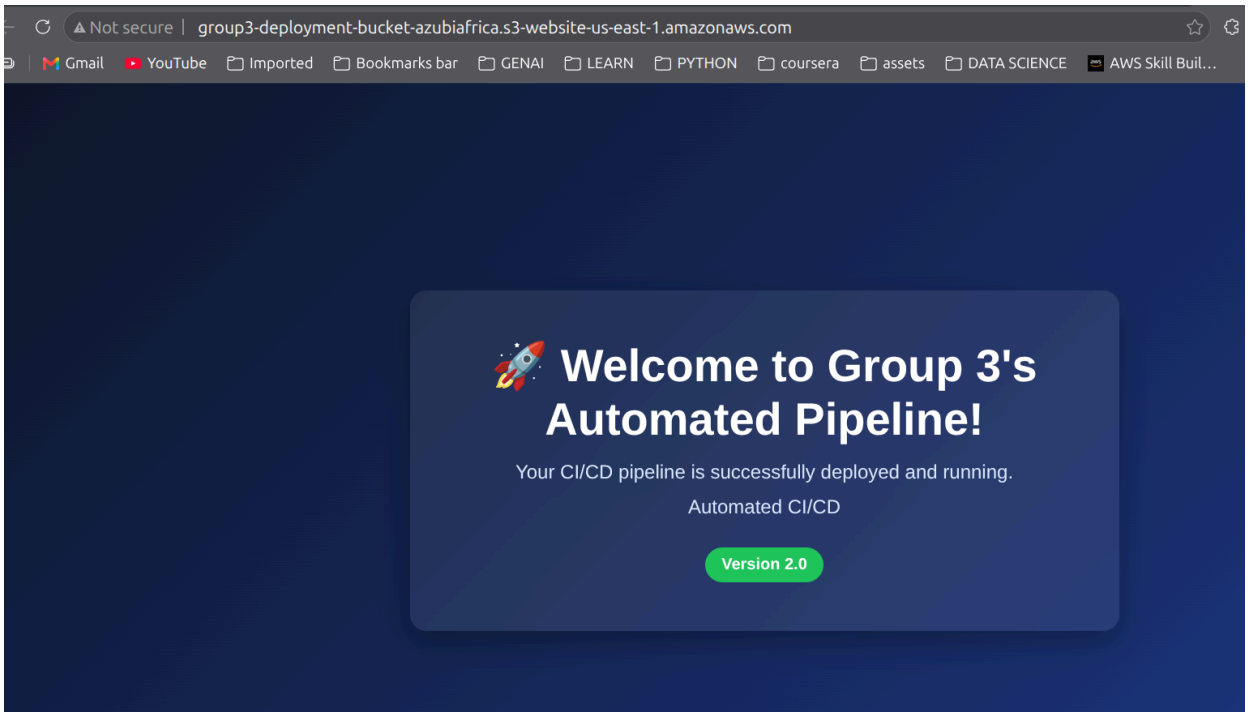


Screenshot showing our initial Version 1.0 website live on the S3 endpoint

Task 5: Test the Pipeline (The Magic of Automation)

To prove that our automation was working flawlessly, we went back to AWS CodeCommit and edited our index.html file. We changed the text to say "Version 2.0" and committed the changes.

We didn't have to touch anything else. The moment we committed the code, CodePipeline detected the change, automatically triggered CodeBuild to compile it, and pushed the new version to S3. We simply refreshed our web browser, and our updated site was live!



Screenshot showing the live website successfully updated to Version 2.0 via automated CI/CD!